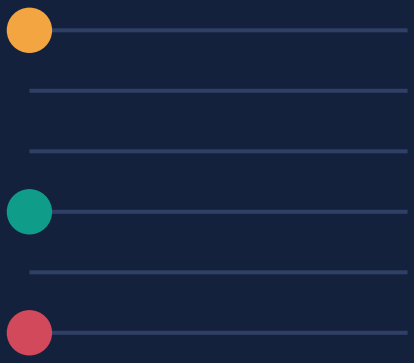

INFORME TÉCNICO | PROCESAMIENTO DE LENGUAJE NATURAL

Detector de noticias falsas con Naive Bayes

Análisis del código, resultados y limitaciones

scikit-learn - CountVectorizer - MultinomialNB

CONTENIDO

01 Resumen ejecutivo
02 Datos y funcionamiento del código
03 Resultados de la ejecución
04 Análisis crítico
05 Mejoras y conclusión

01 RESUMEN EJECUTIVO

Qué hace el código y qué resultado da

El script entrena un clasificador **Naive Bayes multinomial** para distinguir titulares **reales** de **falsos** (*fake news*) a partir de las palabras que contienen. Usa un conjunto de 10 titulares en español, los convierte en conteos de palabras con **CountVectorizer**, reserva el 30% para prueba y evalúa con accuracy, reporte de clasificación y matriz de confusión. Al final clasifica dos titulares nuevos.

100%	3 de 10	64 palabras	87%
Accuracy reportada	Tamaño del test	Vocabulario	Accuracy media (100 cortes)

Conclusión principal. El código es correcto y funciona, pero el 100% de accuracy **no es una medida fiable**: se calcula sobre solo 3 ejemplos y cambia entre 33% y 100% según cómo se haga la partición. Sirve como demostración didáctica del flujo de trabajo, no como detector utilizable.

02 DATOS Y FUNCIONAMIENTO

Cómo está construido el modelo

Conjunto de datos

10 titulares etiquetados a mano, balanceados: **5 'real'** (reforma educativa, hallazgo de agua en Marte, campaña contra el dengue, nuevo hospital, ley ambiental) y **5 'fake'** (vacuna que convierte en robots, Tierra plana, crema milagrosa, chocolate que cura el cáncer, teléfonos que espían sueños).

Flujo paso a paso

Paso	Código	Qué hace
1. Datos	pd.DataFrame(data)	Arma una tabla con las columnas text y label.
2. Vectorizar	CountVectorizer().fit_transform(X)	Pasa a minúsculas, separa palabras (de 2 o más letras) y cuenta cuántas veces aparece cada una. Resultado: matriz de 10 x 64.
3. Partir	train_test_split(..., test_size=0.3, random_state=42)	7 ejemplos para entrenar (3 fake, 4 real) y 3 para probar (2 fake, 1 real).
4. Entrenar	MultinomialNB().fit(...)	Estima P(clase) y P(palabra | clase) con suavizado de Laplace (alpha = 1).
5. Evaluar	accuracy / classification_report / confusion_matrix	Compara las predicciones con las etiquetas reales del test.
6. Predecir	vectorizer.transform(...) + model.predict(...)	Clasifica titulares nuevos con el mismo vocabulario.

Por qué Naive Bayes encaja con texto

Asume que las palabras son independientes entre sí dado la clase y elige la clase que maximiza P(clase) x producto de P(palabra | clase). Es rápido, funciona con pocos datos y se interpreta fácil, aunque la hipótesis de independencia es una simplificación del lenguaje real.

03 RESULTADOS

Qué devolvió la ejecución

Métricas sobre el conjunto de prueba

Clase	Precision	Recall	F1	Soporte
fake	1,00	1,00	1,00	2
real	1,00	1,00	1,00	1
Accuracy			**1,00**	**3**

Matriz de confusión

	Pred. fake	Pred. real
Real: fake	**2**	**0**
Real: real	**0**	**1**

Los tres titulares de prueba fueron clasificados correctamente: el de la vacuna y el de la crema milagrosa como 'fake', y el de la ley ambiental como 'real'.

Clasificación de titulares nuevos

Titular nuevo	Predicción	P(fake)	P(real)	Palabras conocidas por el modelo
Nuevo estudio demuestra que el café mejora la memoria	**fake**	53,5%	46,5%	5 de 9 (nuevo, estudio, que, el, la)
Expertos afirman que los gatos pueden hablar con humanos	**fake**	94,3%	5,7%	3 de 9 (afirman, que, los)

Lectura. El segundo titular sí recibe una probabilidad alta de 'fake' (94,3%), pero el primero queda casi en empate (53,5% contra 46,5%): la etiqueta 'fake' es prácticamente un cara o cruz. El script solo imprime la clase y oculta esa incertidumbre; conviene mostrar `predict_proba`.

04 ANÁLISIS CRÍTICO

Por qué hay que leer el 100% con cautela

Accuracy en 100 particiones aleatorias distintas (random_state = 0..99)

Con solo 3 ejemplos de prueba, cada acierto o error mueve la accuracy 33 puntos. Al repetir el experimento con 100 particiones aleatorias, el 100% aparece en 62 y el resultado cae a 66,7% en 37 y a 33,3% en 1, con un promedio de 87%. Con validación cruzada leave-one-out (10 pruebas de un ejemplo) la accuracy es de 90%. El 100% depende en buena medida de que `random_state=42` sea una partición favorable.

Principales limitaciones

Limitación	Efecto
Muestra mínima (10 titulares)	El modelo memoriza palabras puntuales; no generaliza. Con 64 palabras distintas y 7 ejemplos de entrenamiento hay más variables que datos.
Test de 3 ejemplos	La accuracy es poco informativa: oscila entre 33% y 100% según la partición.
Fuga de datos (data leakage) leve	El vectorizador se ajusta con todos los textos antes de dividir, de modo que el vocabulario incluye palabras del test. Debería ajustarse solo con el entrenamiento.
Sin *stopwords*	Palabras vacías como 'que', 'el', 'la' pesan en la decisión; en los titulares nuevos son buena parte de lo que el modelo reconoce.
Sin estratificación	La partición no garantiza proporciones de clases; con 3 ejemplos de test puede quedar casi sin una de ellas.
Aprende estilo, no verdad	El modelo detecta vocabulario típico de los ejemplos, no si un hecho es cierto. Que 'el café mejora la memoria' salga 'fake' es solo un efecto de las palabras vistas, no un juicio factual.
Sesgo del conjunto	Los falsos son afirmaciones sensacionalistas y los reales, comunicados institucionales; un titular real de estilo llamativo podría clasificarse mal.

05 MEJORAS Y CONCLUSIÓN

Cómo hacerlo más robusto

Recomendaciones

Mejora	Detalle
Más datos	Cientos o miles de titulares etiquetados de fuentes variadas; es lo que más impacto tiene.
Pipeline	Encadenar vectorizador y modelo evita la fuga de datos y simplifica el código.
Validación cruzada estratificada	Reporta media y desvío de la métrica en lugar de depender de una única partición.
TfidfVectorizer + stopwords	Reduce el peso de palabras vacías y resalta las discriminativas.
n-gramas (1, 2)	Captura expresiones como 'confirma el hallazgo' o 'convierte en robots'.
predict_proba	Muestra la confianza de cada predicción y permite un umbral de 'no estoy seguro'.

Ejemplo de versión mejorada

```
from sklearn.pipeline import make_pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.model_selection import cross_val_score, StratifiedKFold

modelo = make_pipeline(
    TfidfVectorizer(ngram_range=(1, 2)),
    MultinomialNB(),
)
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(modelo, df['text'], df['label'], cv=cv)
print(f'Accuracy: {scores.mean():.2%} +/- {scores.std():.2%}')

modelo.fit(df['text'], df['label'])
print(modelo.predict_proba(nuevas_noticias))
```

Nota: con 10 ejemplos, 5 pliegues dejan solo 2 por pliegue; el código está pensado para un conjunto de datos mayor.

Conclusión

El script ilustra correctamente el flujo clásico de clasificación de texto: vectorizar, dividir, entrenar con Naive Bayes, evaluar y predecir. Sus resultados, sin embargo, no demuestran capacidad real de detectar noticias falsas: el conjunto es demasiado pequeño, la evaluación depende de una partición de 3 ejemplos y el modelo se apoya en palabras sueltas. Para un uso real hacen falta más datos, validación cruzada y revisar la confianza de cada predicción; incluso así, un clasificador de este tipo debe verse como apoyo y no como verificador de hechos.